

COS710 - Artificial Intelligence 1: Assignment 3

Seale Rapolai - 15122116

May 2026

1 Dataset

The dataset utilized is the Residential Energy Dataset UK (2014-2020). It contains continuous time-series observations of electricity load demand recorded over a 6-year period. Forecasting electricity consumption is a critical task for energy providers, often addressed through advanced computational methods such as Grammatical Evolution [5]. The original data instances are captured at 15-minute intervals. Given that this is a time-series forecasting problem, past observations must be used to predict future instances. For this experiment, the dataset is restructured through a sliding window by creating lagged attributes; specifically, the models are trained utilizing $m = 8$ contiguous previous values to predict the forthcoming electricity load.

1.1 Preprocessing

The dataset, supplied in CSV format, was loaded into a Python Pandas DataFrame. To maintain temporal integrity, the observations were sorted in ascending order based on the *utc_timestamp* column. *pandas.Timedelta* was then used to verify that the time series was continuous, confirming a uniform 15-minute interval between consecutive data points with no missing entries.

For the feature engineering phase, the *Electricity.load* column was isolated into a new DataFrame to serve as the target variable. To incorporate historical context, the m lagged features were generated by applying shift operations. Specifically, for each $i \in [1, m]$, a new column $load_{t-i}$ was created to represent the electricity load at $t - i$. With m set to 8, these features represent a sliding window of the previous two hours:

- $load_{t-1}$: Load from 15 minutes prior.
- $load_{t-2}$: Load from 30 minutes prior
- Up to $load_{t-8}$ (120 minutes prior).

As a consequence of the shift operations, the initial rows of the dataset contained *null* values where historical data was unavailable. These incomplete rows were removed to ensure a clean dataset for subsequent analysis.

2 Performance metric

The performance metric used to assess the quality of the algorithmic predictor is the **Mean Absolute Percentage Error (MAPE)**.

How it works: MAPE mathematically computes the average absolute percentage difference between the estimated values produced by the decoded phenotype expression tree and the actual known electricity load values in the fitness cases.

Why it was chosen: It is one of the most robust evaluators for numeric regression problems dealing with shifting temporal datasets. Unlike MSE, MAPE provides a scale-invariant metric that prevents the fitness score from being distorted by the naturally fluctuating variance across different seasonal consumption cycles.

3 Standard Grammatical Evolution (GE)

To enforce structural constraints on the evolved predictive models, this assignment utilizes **Standard Grammatical Evolution (GE)** [2].

GE decouples the evolutionary search space from the physical expression space by maintaining a dual representation:

- **Genotype:** A single flat list of integer codons (typically in the range $[0, 255]$). These codons are used to select expansion rules sequentially during the parsing process.
- **Phenotype:** An executable mathematical expression tree generated by recursively parsing the genotype using a context-free grammar.

Parsing begins at the start symbol of the grammar. At each step, the next non-terminal is expanded by retrieving the next codon in the flat list and choosing the production rule index using a modulo operator:

$$\text{Rule Index} = \text{codon} \% \text{Number Of Rules For Non Terminal} \quad (1)$$

If the codon list is exhausted before the derivation terminates, the reading pointer wraps around back to the beginning of the genotype list. A maximum wrap-around limit of 10 wraps (or a maximum node limit of 1000) is enforced to prevent infinite loops. Individuals exceeding these limits are declared invalid and assigned a high penalty fitness value of 10^9 .

4 Representation, Terminal Sets, and Function Sets

The solution uses a grammar-defined symbolic representation. The Backus-Naur Form (BNF) grammar maps the flat search space of integer codons to syntax trees, maintaining the mathematical order of operations while ensuring that the resulting equations are transparent and human-interpretable.

The Context-Free Grammar (CFG) used in this assignment is defined below:

```

⟨root⟩ ::= StructuredRoot(⟨expr⟩, ⟨expr⟩)
⟨expr⟩ ::= Function(⟨op⟩, ⟨expr⟩, ⟨expr⟩) | Variable(⟨var⟩) | Constant(⟨const⟩)
⟨op⟩ ::= + | − | × | ÷ | max | min
⟨var⟩ ::= loadt−1 | loadt−2 | loadt−3 | loadt−4 | loadt−5 | loadt−6 | loadt−7 | loadt−8
⟨const⟩ ::= C

```

4.1 Terminal set:

The terminals consist of the $m = 8$ contiguous lagged values extracted from the historical sliding window (i.e., $load_{t-1}$ to $load_{t-8}$), combined with an ephemeral random constant C to inject variance.

- $load_{t-i}$: Electricity load at i previous time steps, for $i \in [1, 8]$.
- C : An ephemeral constant value derived directly from the codon value using Equation 2, yielding coefficients cleanly in the range $[-0.2, 0.2]$.

$$C = \text{round}(((\text{codon} \% 256) / 255.0) * 0.4 - 0.2, 2) \quad (2)$$

4.2 Function set:

The set consists of basic arithmetic primitives and threshold operators configured as binary functions, with an arity of 2:

- Arithmetic operators: Addition (+), Subtraction (−), Multiplication (×), and safe Division (÷, which returns 1 when the denominator is zero).
- Threshold operators: Minimum (min) and Maximum (max), which capture peak energy loads and seasonal demand limits.

Equation 3 below shows an example of a decoded phenotype formula:

$$\hat{y}_t = \text{StructuredRoot}(\min(load_{t-2}, load_{t-1}) + (-0.12 \times -0.04)) \quad (3)$$

Figure 1 visualizes the phenotype structure corresponding to Equation 3.

5 Initial population generation

The initial population is generated using **Sensible Initialisation** in association with the **Ramped Half-and-Half** method [4] to guarantee valid initial genotypes.

For a given population size, the derivation process employs a 50/50 split between two creation strategies:

- **Full method:** During the recursive grammar derivation, if the current depth is strictly less than the target depth, the expansion rule is locked to index 0 (Function). This forces the tree to grow until it reaches the maximum depth limit, where it terminates.

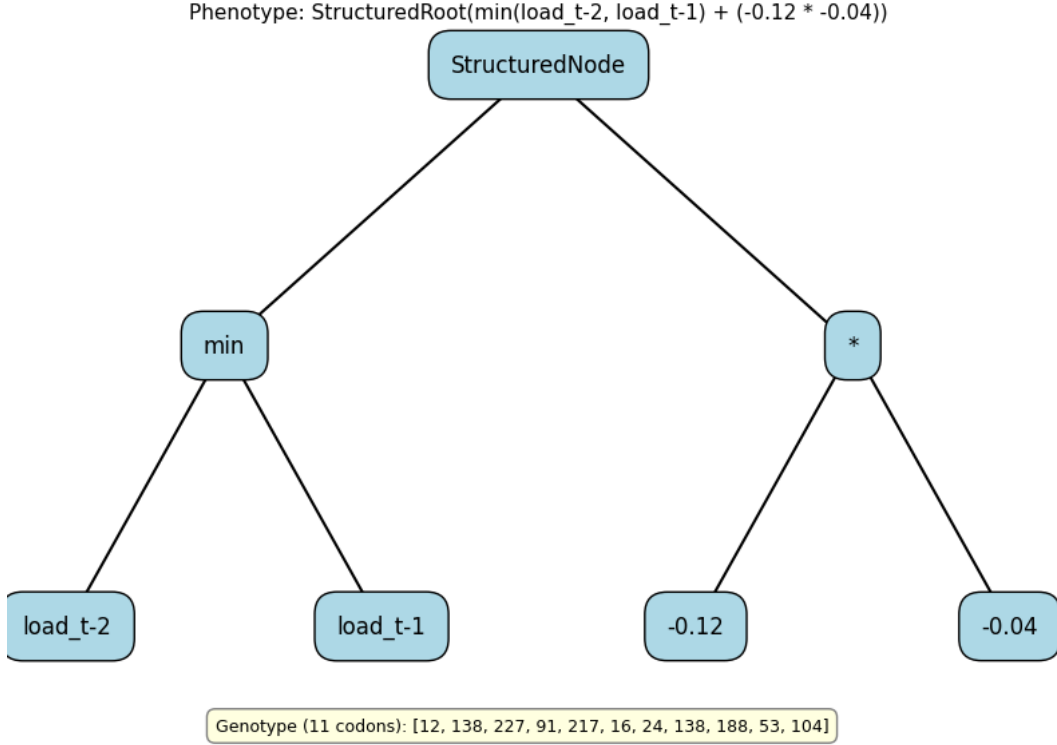


Figure 1: Example phenotype tree representing a grammar-derived individual.

- **Grow method:** While the current depth is less than the target depth, the rule index is selected completely at random from all valid production rules (**Function**, **Variable**, or **Constant**), allowing branches to terminate early and produce asymmetrical trees.

Once the target depth is reached, rule selection is restricted to terminal options (Variable or Constant) to force termination. To map these derivations to the flat list representation, the selected rule indices are mapped to codon values (`codon % len(rules) == r_idx`), generating a standard flat integer array of standard 8-bit integers (`[0, 255]`).

6 Fitness

6.1 Fitness Cases

The algorithm evaluates individuals across a dynamic sliding window extracted from the time-series dataset. A batch window (representing a specific number of days, governed by the `window-size` argument) is selected. Each row within this temporal slice acts as a single fitness case, offering the 8 lagged input variables ($load_{t-1}$ to $load_{t-8}$) and the target `Electricity_load`. The window shifts forward every generation, ensuring the population learns to generalize across seasonal consumption shifts instead of overfitting to a static dataset.

6.2 Fitness Evaluation

During evaluation, each individual’s integer genotype is decoded into an executable phenotype expression tree. The raw fitness is computed using the scale-invariant **Mean Absolute Percentage Error (MAPE)**.

The exact fitness function $F(T)$ for a decoded phenotype tree T evaluated over N fitness cases is defined in Equation 4:

$$F(T) = \frac{1}{N} \sum_{i=1}^N \begin{cases} 10 & \text{if } T(x_i) \text{ results in an exception} \\ \left| \frac{y_i - T(x_i)}{\max(|y_i|, 10^{-8})} \right| & \text{otherwise} \end{cases} \quad (4)$$

where N is the number of fitness cases, x_i is the input lagged vector, $T(x_i)$ is the model’s predicted output, and y_i is the actual target load.

To ensure algorithmic stability and dynamic modeling, the evaluation pipeline includes three critical safeguards:

1. **Denominator Bounding:** The term $\max(|y_i|, 10^{-8})$ prevents division-by-zero errors when the target load y_i is zero or infinitesimally small. By clamping the denominator to a microscopic lower bound (10^{-8}), the function mathematically guarantees a finite MAPE score regardless of anomalous zero-values in the dataset.
2. **Exception Handling:** If the phenotype equation triggers a mathematical exception (e.g., extreme overflow) on a specific test case x_i , that individual case is assigned a severe localized penalty of 10 (equivalent to a 1000% error). This aggressively discourages mathematically unstable structures.
3. **Structural Parsimony Pressure:** To prevent the algorithm from finding a trivial static optimum (constant bloat) and to explicitly control unchecked code growth [7], the fitness evaluator actively inspects the phenotype tree structure prior to calculation. If the decoded equation is entirely devoid of lagged **Variable** nodes (e.g., it is a flat baseline constant), it is immediately rejected and assigned a penalty MAPE of 1.0. This ensures the evolutionary process is strictly forced to model dynamic relationships instead of settling on safe averages.

Finally, if an individual fails the parsing phase entirely (e.g., by exceeding the grammar wrap-around limit and failing to produce a valid tree), it is immediately assigned an extreme global penalty of 10^9 . Because the evolutionary loop actively minimizes $F(T)$ toward zero, these penalty mechanisms guarantee that invalid and structurally unstable genotypes are swiftly eliminated from the breeding pool.

7 selection method

The algorithm utilizes **Tournament Selection** [8] to select individuals for genetic operators. For each selection event, the algorithm samples a uniform subset of individuals from the population (defined by the `tournament-size` parameter). These candidates compete, and the individual with the lowest phenotype MAPE wins the tournament and is passed to the genetic operators.

Tournament selection provides several key advantages:

1. **Scale Independence:** By comparing only the relative rank within the tournament, the selection is immune to large fluctuations in absolute MAPE values across shifting seasonal windows.
2. **Adjustable Selection Pressure:** Tuning the tournament size alters the convergence velocity. A small size (e.g., 2) maintains genetic diversity by allowing weaker genotypes to occasionally breed, whereas a larger size drives rapid convergence.
3. **High Efficiency:** Selecting parents operates in $O(k)$ time (where k is the tournament size), bypassing the computational overhead of global sorting.

8 Genetic operators

The standard GE algorithm applies three core genetic operators at the genotype level to navigate the flat search space: **Crossover**, **Mutation**, and **Reproduction**. These operators are applied based on strict proportional rates.

1. **Reproduction (Elitism):** The top percentage of the population is identified by sorting the fitness values. Their genotype lists are copied directly to the next generation without modification. This operator heavily promotes **exploitation**. By preserving the mathematically fittest models exactly as they are, it ensures the algorithm does not regress and safely maintains the best discovered solutions across generations.
2. **Flat Genotype Crossover:** Two parents are chosen via Tournament Selection. A random single-point cut is chosen within their overlapping length, and their flat integer lists are recombined to generate two child genotypes. If the decoded children violate the global `max_depth` constraint, the crossover is rejected and the parents are returned unaltered. This operator perfectly balances both search strategies. It **exploits** known fit sub-components by combining successful genetic sequences, while simultaneously **exploring** novel structural configurations in the phenotype tree. Additionally, rejecting invalid children safely curbs uncontrolled structural bloat.
3. **Flat Genotype Mutation:** Codon flip mutation is applied to a single parent: every codon has a 15% chance of being replaced by a completely new random integer in $[0, 255]$. This seemingly high per-codon mutation rate is intentionally chosen to counteract the genetic redundancy inherent in Grammatical Evolution. Because multiple integers map to the same grammar rule via the modulo operator, many codon flips result in neutral mutations that do not alter the phenotype. The 15% rate ensures that enough meaningful structural changes actually survive the mapping process to drive evolution. This operator acts as the primary engine for **exploration**, introducing entirely fresh genetic material into the population to prevent premature stagnation and escape local optima. Furthermore, fine-tuning is achieved organically during this exploratory phase, as mutating a constant codon directly shifts its derived floating-point coefficient.

8.1 Structural Similarity in Crossover

To prevent redundant crossover events that lead to local optima, the system implements morphological similarity tracking using two distinct indices computed on the decoded phenotype trees [1]:

- **Global Index (GI):** Measures the number of matching function nodes shared between two phenotypes from the root down to a cutoff depth limit (half of the maximum depth).
- **Local Index (LI):** Counts matching function and terminal nodes in the subtrees beyond the cutoff depth.

9 Experimental setup

Hardware specifications: Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz with 16 GB of RAM.

Table 1 lists the parameters applied during this simulation:

Parameter	Value
Population size	100
Max depth	5
Initial population generation	Sensible Ramped half-and-half
Fitness function	<i>MeanAbsolutePercentageError(MAPE)</i>
Test Cases / Window Constraints	30 days sliding window chunk offsets
Selection method	Tournament Selection
Tournament Size	2
Mutation rate	19% (19 individuals per 100)
Crossover rate	80% (80 individuals per 100)
Reproduction Rate	1% (1 individual per 100)
Number of generations	70
Termination criteria	Fixed generation limit reached

Table 1: Standard GE configuration parameters

The parameters balance computational efficiency, search space traversal, and phenotypic interpretability:

- **Population Size (100) & Max Depth (5):** Prevents tree bloat and ensures the evolved equations are lean, highly interpretable, and generalized.
- **Genetic Rates (80% Crossover / 19% Mutation / 1% Elitism):** Maximizes constructive recombination while continuously introducing structural and constant-level variance.
- **Tournament Size (2):** Maintains diverse selection dynamics, allowing suboptimal but structurally novel individuals to breed.

- **Generations (70):** Explicitly calculated to cover the entire historical dataset (201,596 instances) over the sliding window length of 30 days (2,880 instances per generation):

$$\text{Generations} = \left\lceil \frac{\text{Total Instances}}{\text{Instances per Window}} \right\rceil = \left\lceil \frac{201,596}{2,880} \right\rceil \approx 70 \quad (5)$$

10 Results

The Standard GE algorithm successfully converged across all experimental runs. The system demonstrated strong generalization capacities, navigating the tumbling sliding window that exposes the population to new temporal offsets every generation.

Run NO.	Random seed	Best Individual (MAPE)
1	35	0.1996
2	83	0.2008
3	75	0.2101
4	97	0.2107
5	14	0.2146
6	40	0.2150
7	66	0.2160
8	36	0.2187
9	89	0.2258
10	47	0.2348
mean		0.2146
Std. Div		0.0100

Table 2: Execution results for the best 10 runs

As shown in Table 2, across 10 independent experimental runs, the best individual consistently achieved a MAPE between **19.96%** and **23.48%**. The mean across all runs is **21.46%**, with a narrow standard deviation of **1.00%**, highlighting the algorithm’s robustness across multiple independent runs with different random seeds.

The best individual across all runs was found in **Run 1 (Seed 35)**, with a MAPE of **19.96%**. The evolved phenotype heavily utilizes nested arithmetic and variable structures to model peak loads. Figure 2 illustrates this tree.

Due to the online sliding window, the algorithm experiences minor dataset shock at the start of each generation. However, the average and worst errors converge sharply as shown in Figure 3, crashing downward from extreme generation-0 outliers and squeezing tightly against the elite baseline.

Figure 4 exhibits minor fluctuations (bouncing between roughly 12% and 20% MAPE), which is an expected consequence of the tumbling sliding window. Every generation, the elite tree is evaluated on unseen data, reflecting shifting seasonal predictability. The stable elite MAPE confirms the discovery of highly generalized models that resist overfitting.

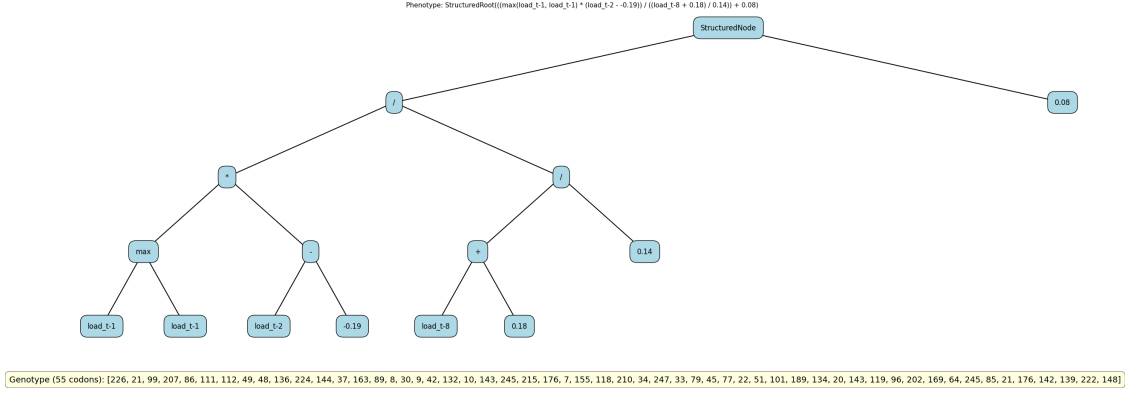


Figure 2: Binary Expression Tree of the best individual from run 1, using 35 as random seed.

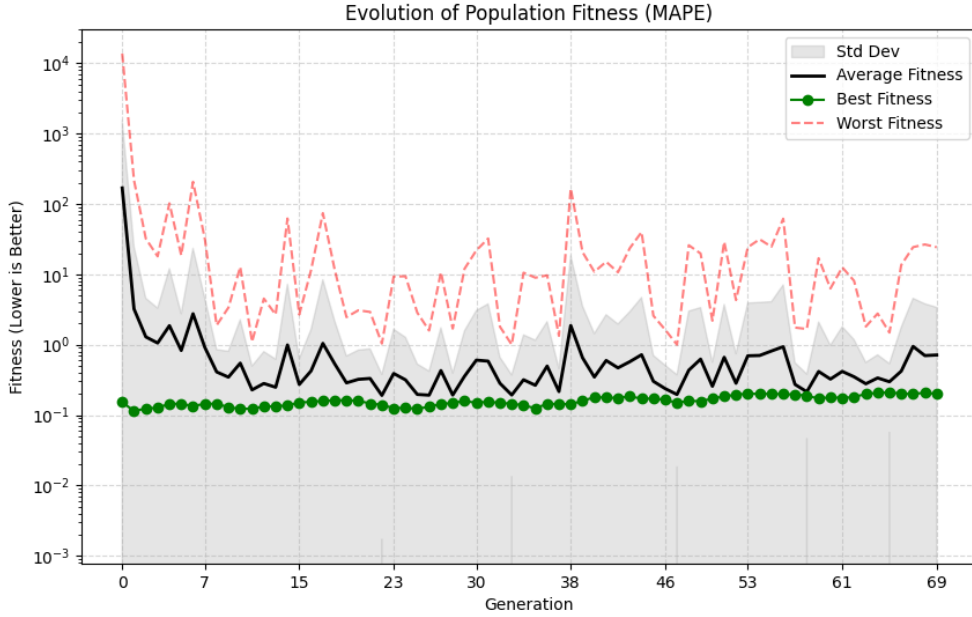


Figure 3: Evolution of population fitness over generations.

11 Run times

To ensure computational scalability across the 5.7 years of temporal data, the fitness evaluation pipeline was fully refactored to utilize asynchronous multiprocessing, distributing the load of evaluating 100 individuals across all available CPU cores.

Across the 10 experimental runs, processing 70 generations took between **870.32 seconds** (14.5 minutes) and **951.00 seconds** (15.8 minutes). On average, a complete traversal of the 201,596-row dataset took roughly **15.0 minutes**.

The run times shown in Table 3 prove that the algorithm successfully balances structural complexity with processing speed, maintaining a relatively lean tree architecture (Max Depth = 5) while computing tens of millions of mathematical operations across the feature set. The overall mean execution time of **897.53 seconds** (approximately

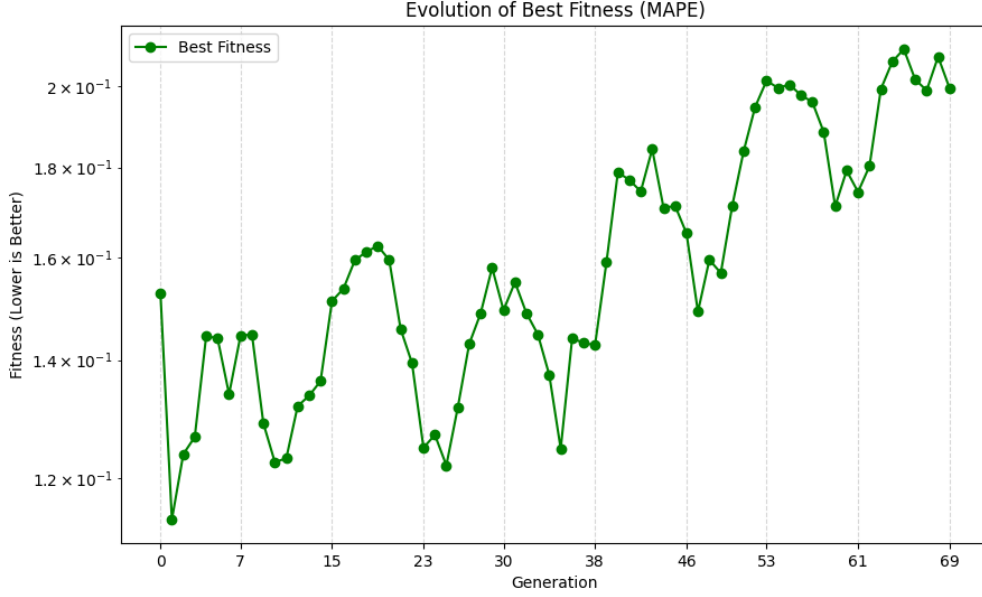


Figure 4: Evolution of best individual’s fitness per generation.

15.0 minutes) demonstrates highly efficient traversal of the 5.7-year dataset. The standard deviation of **25.44 seconds** indicates that the computational load is extremely stable across different random seeds, successfully mitigating the extreme processing variances commonly caused by uncontrolled tree bloat.

12 Comparison of Previous Assignments

The fundamental evolution from Assignment 1 and 2 to Assignment 3 lies in the mechanism of structural enforcement: the transition from Genetic Programming (GP) to Standard Grammatical Evolution (GE). This architectural upgrade completely redefines how structural validity is maintained during the evolutionary process.

12.1 Context-Free Grammar as the Structural Core

In standard GP (Assignment 1) and Structure-Based GP (Assignment 2), the search space and the expression space were identical: operations were performed directly on the morphological trees. While Assignment 2 utilized a similarity index to manage structural diversity, it still relied on tree-based crossover which inherently risked creating invalid or bloated mathematical expressions if not strictly constrained.

In contrast, Assignment 3 introduces the **Context-Free Grammar (CFG)** and **Genotype-Phenotype mapping** as its primary structural component. By decoupling the flat integer search space (genotype) from the executable trees (phenotype), GE offloads all structural responsibilities to the grammar.

Run NO.	Random seed	Execution Time (Seconds)
1	35	888.09
2	83	923.20
3	75	924.04
4	97	951.00
5	14	884.55
6	40	872.59
7	66	870.32
8	36	883.82
9	89	876.71
10	47	901.02
mean		897.53
Std. Div		25.44

Table 3: Execution results for the best 10 runs

12.2 Enforcing Structural Validity and Constraints

The grammar serves as an absolute structural blueprint. It explicitly enforces the order of operations, prevents mathematical type mismatches, and restricts the search space exclusively to valid equations. Because crossover and mutation occur on the flat integer lists, the genetic operators are completely agnostic to the phenotype structure.

However, the integrity of the resulting phenotype is mathematically guaranteed by the CFG during the decoding phase. This strict, decoupled structure mitigates the destructive crossover events seen in Assignment 1. Furthermore, while the GE architecture inherently manages structural validity, the algorithm strategically retains the morphological similarity tracking (Global and Local Indices) from Assignment 2 to prevent redundant crossover events, blending the structural safety of GE with the diversity-preservation of Structure-Based GP.

12.3 Comparison of Execution Metrics and Performance

The architectural differences between the three assignments directly impacted their convergence accuracy and computational efficiency.

Computational Efficiency and Run Times: Assignment 1 (Standard GP) suffered from unconstrained code growth (bloat) and expensive direct tree recombinations. It required approximately 12 to 13 minutes to simulate just 20 generations. Scaled to the 70 generations used in later assignments, it would be highly inefficient. Assignment 2 (Structure-Based GP) solved the bloat issue by introducing morphological similarity tracking (Global and Local Indices). While this allowed the system to simulate 70 generations in an average of **20.5 minutes**, the recursive tree-comparison algorithms introduced significant overhead. Assignment 3 (Standard GE) proved to be the most computationally efficient architecture, completing 70 generations in a mean time of just **15.0 minutes**. By shifting genetic operations (crossover and mutation) to flat integer arrays rather than recursive tree structures, the genetic operators execute in linear time. Although it retains the morphological similarity checks from Assignment 2 to prevent redundant crossover, the strict structural parsimony pressure limits tree bloat

so effectively that the overhead of these similarity checks remains negligible, leading to massive overall execution speedups.

Predictive Accuracy (MAPE): Because Assignment 1 utilized Mean Squared Error (MSE), its raw fitness values cannot be directly compared to the latter iterations. However, Assignments 2 and 3 both utilize the Mean Absolute Percentage Error (MAPE) under identical tumbling window constraints. Assignment 2 achieved a slightly superior mean MAPE of **19.10%**, whereas Assignment 3 achieved **21.46%**. This minor regression in accuracy is a known trade-off of Grammatical Evolution. Because GE utilizes an indirect genotype-to-phenotype mapping via modulo operations, a single point mutation in the integer array can drastically alter the decoding pathway of all subsequent codons (the ripple effect). This makes fine-tuning (exploitation) slightly harder in GE than in the direct tree-based mutations of Structure-Based GP, resulting in a marginally higher final error rate despite the massive gains in execution speed and structural safety.

References

- [1] Kapoor, R., & Pillay, N. (2023). Iterative Structure-Based Genetic Programming for Neural Architecture Search. *Proceedings of the Companion Conference on Genetic and Evolutionary Computation (GECCO '23 Companion)*, 595–598.
- [2] O'Neill, M., & Ryan, C. (2001). Grammatical evolution. *IEEE Transactions on Evolutionary Computation*, 5(4), 349–358.
- [3] Montana, D. J. (1995). Strongly Typed Genetic Programming. *Evolutionary Computation*, 3(2), 199–230.
- [4] Poli, R., Langdon, W. B., & McPhee, N. F. (2008). *A Field Guide to Genetic Programming*. Lulu.com.
- [5] Castelli, M., Vanneschi, L., & Silva, S. (2015). Prediction of electricity consumption by using genetic programming with semantic-based genetic operators. *Statistical Analysis and Data Mining*, 8(4), 242–256.
- [6] Makridakis, S. (1993). Accuracy measures: theoretical and practical concerns. *International Journal of Forecasting*, 9(4), 527–529.
- [7] Luke, S., & Panait, L. (2006). A comparison of bloat control methods for genetic programming. *Evolutionary Computation*, 14(3), 309–344.
- [8] Miller, B. L., & Goldberg, D. E. (1995). Genetic algorithms, tournament selection, and the effects of noise. *Complex Systems*, 9(3), 193–212.